Malware Analysis — 178ba564b39bd07577e974a9b677dfd86ffa1f1d0299dfd958eb883c5ef6c3e1.exe

Category: no category supported by the evidence
Triage hint: MALICIOUS (65/100, basis: dynamic-confirmed); a screening flag only
Family: loader/packer
Completeness: static + dynamic + detonation
SHA-256: `178ba564b39bd07577e974a9b677dfd86ffa1f1d0299dfd958eb883c5ef6c3e1`
Type: PE32 / x86 · 212,992 bytes
Report generated: 2026-10-21 16:31:37

Contents: 1) Analyst report · 2) Appendix A — Methodology & reproducibility · 3) Appendix B — Proof of compromise

---

Malware Analysis Report — 178ba564b39bd07577e974a9b677dfd86ffa1f1d0299dfd958eb883c5ef6c3e1.exe

Triage hint: MALICIOUS (65/100 · basis: dynamic-confirmed). This is a quick screening flag, not the purpose of this report: the analysis below (category, stages, dumps, indicators, encryption, author leads) is the result.
Category: no category supported by the evidence
Family (behavioral): loader/packer
Completeness: static + dynamic + detonation
SHA-256: `178ba564b39bd07577e974a9b677dfd86ffa1f1d0299dfd958eb883c5ef6c3e1`
Size: 212,992 bytes

1. Executive summary

- Identified as **PE32 / Microsoft Visual C/C++ 19.00.24215** (x86).
- Detonated live in an isolated instance for 90s; behavior, network, and a process memory dump were captured.

Why the verdict is MALICIOUS (65/100, basis: dynamic-confirmed): *Dynamic evidence (what it actually did — subtotal 65):*

- CONFIRMED self-unpacking at runtime (payload captured + verified) (+30)
- runtime: memory made executable after being written (2 W->X event(s)) — the unpacking moment (+10)
- behavioral classification: loader/packer (loader/packer) (+25)

1 · IOC — Indicators of Compromise (attacker communication / C2)

- No network IOCs recovered yet. For this sample the C2 config is likely still encrypted (decrypted at beacon time) — see the Encryption section for how to recover it, or run the detonation phase to capture live callbacks.

1a. Author / build reconnaissance

- Compiled 2020-04-24 02:04:11 UTC
- Version info: CompanyName=INCA Internet Co., Ltd., LegalCopyright=Copyright (C) INCA Internet. 2000-2003, Comments=nProtect KeyCrypt Program Database DLL, OriginalFilename=npkpdb.dll, ProductName=nProtect KeyCrypt Program Database DLL, InternalName=npkpdb.dll
- MSVC toolchain build IDs [21005, 21022, 23026, 23918, 24210, 24213, 24215, 25834, 30319, 30501, 30729, 31101, 40219, 40629, 50727, 60610, 61030] (Rich-header fingerprint for same-author clustering)
- Full version-info resource:
  - Comments: `nProtect KeyCrypt Program Database DLL`
  - CompanyName: `INCA Internet Co., Ltd.`
  - FileDescription: `nProtect KeyCrypt Program Database DLL`
  - FileVersion: `2003, 10, 1, 1`

- InternalName: `npkpdb.dll`
- LegalCopyright: `Copyright (C) INCA Internet. 2000-2003`
- OriginalFilename: `npkpdb.dll`
- ProductName: `nProtect KeyCrypt Program Database DLL`
- ProductVersion: `4, 0, 0, 0`

Author / build fingerprints and clustering

File	imphash	Rich-header hash	Compile time (UTC)	Signed
`api_fullmodule_0x10000000_PE.bin`	`eb76140cc31cd615e218e51dbcad0a7b`	`55df3578182b8514a73410fe2fc7c63c`	2020-04-16 00:47	False
`api_fullmodule_0x10000000_rebuilt.exe`	`eb76140cc31cd615e218e51dbcad0a7b`	`55df3578182b8514a73410fe2fc7c63c`	2020-04-16 00:47	False
`api_region1_0x1e0000_PE.bin`	`d073d1854680ff2ddd038760f5090798`	`495b4d33a880c7a6330c31b841e8035f`	2020-04-23 12:36	False
`api_region1_0x1e0000_rebuilt.exe`	`d073d1854680ff2ddd038760f5090798`	`495b4d33a880c7a6330c31b841e8035f`	2020-04-23 12:36	False
`packed sample`	`7ebef1081a372c6e6f289d7c633f4f3a`	`a08e6e4e71a4e9438fd0f44de4641ca7`	2020-04-24 02:04	False

Shared fingerprints with earlier analysed cases (a lead for same builder/author/toolchain, not proof):

- **strong lead**: `api_fullmodule_0x10000000_PE.bin` shares imphash, rich_hash, timestamp with `api_fullmodule_0x10000000_PE.bin` in case `710ada92-23df-45ee-82e1-84e402c7901a_dridex`
- **strong lead**: `api_fullmodule_0x10000000_PE.bin` shares imphash, rich_hash, timestamp with `api_fullmodule_0x10000000_rebuilt.exe` in case `710ada92-23df-45ee-82e1-84e402c7901a_dridex`
- **strong lead**: `api_fullmodule_0x10000000_rebuilt.exe` shares imphash, rich_hash, timestamp with `api_fullmodule_0x10000000_PE.bin` in case `710ada92-23df-45ee-82e1-84e402c7901a_dridex`
- **strong lead**: `api_fullmodule_0x10000000_rebuilt.exe` shares imphash, rich_hash, timestamp with `api_fullmodule_0x10000000_rebuilt.exe` in case `710ada92-23df-45ee-82e1-84e402c7901a_dridex`
- **strong lead**: `api_region1_0x1e0000_PE.bin` shares imphash, rich_hash, timestamp with `api_region1_0x1e0000_PE.bin` in case `710ada92-23df-45ee-82e1-84e402c7901a_dridex`
- **strong lead**: `api_region1_0x1e0000_PE.bin` shares imphash, rich_hash, timestamp with `api_region1_0x1e0000_rebuilt.exe` in case `710ada92-23df-45ee-82e1-84e402c7901a_dridex`
- **strong lead**: `api_region1_0x1e0000_rebuilt.exe` shares imphash, rich_hash, timestamp with `api_region1_0x1e0000_PE.bin` in case `710ada92-23df-45ee-82e1-84e402c7901a_dridex`
- **strong lead**: `api_region1_0x1e0000_rebuilt.exe` shares imphash, rich_hash, timestamp with `api_region1_0x1e0000_rebuilt.exe` in case `710ada92-23df-45ee-82e1-84e402c7901a_dridex`
- **strong lead**: `packed sample` shares imphash, rich_hash, timestamp, section_names with `packed sample` in case `710ada92-23df-45ee-82e1-84e402c7901a_dridex`

1b. Behavioral classification

Behaves as a **loader/packer**.

Family	Score	Evidence
loader/packer	8	allocates memory then flips it W->X (self-unpacking); makes written memory executable at runtime; resolves APIs at runtime via GetProcAddress/LoadLibrary

1c. Dynamic unpack (x64dbg, debugger-driven)

- **Payload captured: YES** — the packer decrypted its code in memory and flipped it writable→executable (2 W→X flip event(s)); we dumped it at that instant (see the Memory dumps section).

Unpack verification: CONFIRMED_UNPACKED (60/100)

File	Size	MD5	SHA-256
packed (input): packed sample	212992	6a8401448a5bd2b540850f811b20a66d	178ba564b39bd07577e974a9b677dfd86ffa1f1d0299dfd958eb883c5ef6c3e1
unpacked (output): reconstructed: api_fullmodule_0x10000000_rebuilt.exe	217088	9fc8d8fc35a3e4eb385f169eb16d6d16	bc0886d626ba6ca8c60ab997b578c66a92abae0a72fa1a17fbe4cd0570892289

Compare these MD5s against your file set — a packer that writes copies of itself will show the SAME hash across dropped files.

Score breakdown (why this verdict):

Signal	Points	Detail
W->X decryption captured	50/50	debugger captured the payload at 2 write->execute flip(s) (direct unpack evidence)
import table grew	0/35	imports 17 -> 7 (no growth)
packer sections removed	0/25	no packer section names in either
entropy dropped	0/20	entry-point entropy 5.55 -> 6.55 (no significant drop)
hidden IOCs revealed	0/25	no new network IOCs in the dump (may be encrypted config)
valid PE image	10/10	dump parses as a valid PE image

1e. API trace (live dynamic behavior)

Watched the P1/P2 API arsenal during execution. Call counts:

API	Calls
`kernel32.VirtualProtect`	6
`ntdll.NtProtectVirtualMemory`	6
`kernel32.VirtualAlloc`	3
`kernel32.LoadLibraryA`	3

Notable calls (with captured arguments / memory):

- `kernel32.VirtualAlloc`(lpAddress=0x0, dwSize=0x2b000, flType=0x1000, flProtect=0x4) — returned base 0x670000
- `kernel32.VirtualAlloc`(lpAddress=0x0, dwSize=0x6000, flType=0x1000, flProtect=0x40) — returned base 0x1e0000
- `kernel32.LoadLibraryA`(lpName=0x1e30f2) — ””
- `kernel32.VirtualAlloc`(lpAddress=0x0, dwSize=0x2a000, flType=0x1000, flProtect=0x4) — returned base 0x6f0000
- `kernel32.LoadLibraryA`(lpName=0x717e80) — ”KERNEL32.dll”
- `kernel32.LoadLibraryA`(lpName=0x717e9e) — ”ADVAPI32.dll”
- `kernel32.VirtualProtect`(lpAddress=0x10000000, dwSize=0x35000, flNewProtect=0x4, lpOld=0x19fda0) — protect 0x4 (not exec)
- `ntdll.NtProtectVirtualMemory`(hProc=0xffffffff, pBase=0x19fcf8, pSize=0x19fcfc, newProt=0x4) — NtProtect newProt=0x4
- `kernel32.VirtualProtect`(lpAddress=0x10000000, dwSize=0x400, flNewProtect=0x2, lpOld=0x19fda0) — protect 0x2 (not exec)
- `ntdll.NtProtectVirtualMemory`(hProc=0xffffffff, pBase=0x19fcf8, pSize=0x19fcfc, newProt=0x2) — NtProtect newProt=0x2
- `kernel32.VirtualProtect`(lpAddress=0x10001000, dwSize=0x20d11, flNewProtect=0x20, lpOld=0x19fda0) — W->X region 0x10001000+0x20d11 [full-module dumped] [dumped]

- `ntdll.NtProtectVirtualMemory`(hProc=0xffffffff, pBase=0x19fcf8, pSize=0x19fcfc, newProt=0x20) — W->X (native) 0x10001000+0x20d11 [dumped]
- `kernel32.VirtualProtect`(lpAddress=0x10022000, dwSize=0x5efe, flNewProtect=0x2, lpOld=0x19fda0) — protect 0x2 (not exec)
- `ntdll.NtProtectVirtualMemory`(hProc=0xffffffff, pBase=0x19fcf8, pSize=0x19fcfc, newProt=0x2) — NtProtect newProt=0x2
- `kernel32.VirtualProtect`(lpAddress=0x10028000, dwSize=0x3e8, flNewProtect=0x4, lpOld=0x19fda0) — protect 0x4 (not exec)
- `ntdll.NtProtectVirtualMemory`(hProc=0xffffffff, pBase=0x19fcf8, pSize=0x19fcfc, newProt=0x4) — NtProtect newProt=0x4
- `kernel32.VirtualProtect`(lpAddress=0x10029000, dwSize=0xec8, flNewProtect=0x2, lpOld=0x19fda0) — protect 0x2 (not exec)
- `ntdll.NtProtectVirtualMemory`(hProc=0xffffffff, pBase=0x19fcf8, pSize=0x19fcfc, newProt=0x2) — NtProtect newProt=0x2

Allocated regions watched: `0x670000`, `0x1e0000`, `0x6f0000`

1f. PE reconstruction (memory dump -> valid PE, IAT rebuilt)

Memory dumps were rebuilt into valid PEs (align raw<->virtual + fix ImageBase), so the import table/IAT resolve:

- Best image: `api_fullmodule_0x10000000_rebuilt.exe` from `api_fullmodule_0x10000000.bin` -> **7 imports across 2 DLLs**
- DLLs: `ADVAPI32.dll`, `KERNEL32.dll`

1 · AI — Analyst assessment (AI-assisted interpretation)

*AI reasoning over the extracted evidence. Advisory: it guides analysis and strategy; it does **not** set the maliciousness score (that is the deterministic, evidence-gated verdict above).*

Stage: packed — model `gpt-4o`

- **Likely family:** unknown (confidence: low)
- **AI claim — encrypted region:** .text1 (encrypted) — supported: section entropy .text=5.548, .text1=7.958
- **Capabilities:**
  - anti-debugging (int 2Dh)
  - timing checks (rdtsc)
  - environment detection (cpuid)
- **Anti-analysis expected:**
  - anti-debugging (int 2Dh, int3)
  - timing checks (rdtsc)
  - environment detection (cpuid)
- **Suggested dynamic strategy:**
  - Set breakpoints on indirect calls at 0x10003476 and 0x10003497
  - Monitor memory writes to .text1 section
  - Dump memory after initial unpacking routine completes
- **Packer:** likely packed or crypted due to high entropy in .text1 and known packer section name
- **Reasoning:** The presence of a high-entropy section (.text1) and known packer indicators suggest the binary is packed. Anti-analysis techniques such as int 2Dh, rdtsc, and cpuid are present, indicating attempts to evade debugging and virtualized environments. The indirect calls and high entropy suggest runtime unpacking, necessitating dynamic analysis to reveal the payload.

Stage: unpacked — model `gpt-4o`

- **Likely family:** unknown (confidence: low)
- **Encrypted region (unaudited):** .text1 — encrypted (high entropy 7.96)
- **Encrypted region (unaudited):** .rdata — compressed (high entropy 7.982)

- **Encrypted region (unaudited):** .data — compressed (high entropy 7.631)
- **Capabilities:**
  - timing check (rdtsc) at 0x25afe
  - anti-disassembly (int3 sled) at 0x1012
- **Notable strings:**
  - nProtect KeyCrypt Program Database DLL (potential misdirection)
- **Anti-analysis expected:**
  - anti-disassembly (int3 sled)
  - timing check (rdtsc)
- **Suggested dynamic strategy:**
  - Breakpoint on rdtsc to monitor timing checks
  - Monitor memory writes in .text1 section
  - Dump memory after decryption routine in .text1
- **Packer:** not packed, but contains high-entropy sections suggesting encryption/compression
- **Reasoning:** The presence of high-entropy sections (.text1, .rdata, .data) suggests encryption or compression, despite the file not being packed. Anti-analysis techniques such as rdtsc timing checks and int3 sleds indicate attempts to evade debugging. The nProtect KeyCrypt string may be a false flag, as the file's behavior does not align with legitimate software.

AI dynamic plan (static → dynamic hand-off)

The AI proposed this from the static findings; a validator decides what the debugger may act on (APIs must be in our arsenal, regions inside the image). Anti-evasion items are advisory only.

- **Hypothesis:** The sample is likely packed or encrypted using a custom packer, indicated by the high entropy in the .text1 section and known packer section name.
- **Priority API (armed, raised noise cap):** `VirtualAlloc` — Commonly used for allocating memory for unpacked code.
- **Priority API (armed, raised noise cap):** `VirtualProtect` — Used to change memory protection, often seen in unpacking routines.
- **Priority API (armed, raised noise cap):** `NtWriteVirtualMemory` — May be used to write unpacked code into allocated memory.
- **Priority API (armed, raised noise cap):** `FlushInstructionCache` — Ensures CPU executes the latest code, often used after unpacking.
- **Watch region (dumped at end of trace, justified):** rva `0x7000` size `0x2bd6b` — High-entropy section likely contains packed/encrypted code. *(evidence: overlaps a high-entropy high-entropy (H=7.955))*
- **Anti-evasion (advisory):** anti-debugging (int 2Dh, int3) → Use a debugger that can handle these interrupts.
- **Anti-evasion (advisory):** timing checks (rdtsc) → Patch or emulate rdtsc to avoid detection.
- **Anti-evasion (advisory):** environment detection (cpuid) → Emulate or patch cpuid to return expected values.
- **Expected at runtime:** Memory allocation and protection changes for unpacked code.
- **Expected at runtime:** Decrypted or unpacked code written to memory.
- **Expected at runtime:** Execution of unpacked code after cache flush.
- **Plan regions actually dumped:** `0x7000`

2 · DEEP — Deep static analysis (strings re-examined at every stage)

Strings and indicators are extracted from the on-disk file, then **again after unpacking**, then **again after decryption** — unencrypted leads (C2, mutexes, paths, config) typically appear only in the later stages.

Stage 1 — Packed (on-disk file)

- Size 212,992 bytes · entropy **7.536** · 817 strings extracted
- **Anti-analysis indicators** (2): `FindWindow`, `Sleep`

Stage 2 — Unpacked (reconstructed payload)

- Size 217,088 bytes · entropy **5.744** · 338 strings extracted

- **Anti-analysis indicators** (2): `OutputDebugString`, `Sleep`
- **Base64 blobs** (1): `ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789`

Stage 3 — Decrypted

- Not available — no decrypted artifact was produced this run.

2 · ENT — Entropy map (where the encrypted / compressed bytes are)

File entropy **7.536** · image base `0x10000000` · size of image `217088` · entry RVA `0x33f3`

Section	RVA	Raw size	Virtual size	Entropy	Notes
`.text`	0x1000	12,288	9,506	5.548	X
`.rdata`	0x4000	4,096	3,204	3.211	
`.data`	0x5000	4,096	4,292	0.08	W
`.text1`	0x7000	180,224	179,563	7.958	W
`.rsrc`	0x33000	4,096	1,816	1.322	
`.reloc`	0x34000	4,096	276	0.665	

High-entropy regions (likely encrypted or compressed):

Kind	Section	RVA	File offset	Size	Entropy
high-entropy	.text1	0x8000	0x7000	172,032	7.955

Largest resources: 16/1 1,100B H=3.412

2 · CODE — Suspicious code & byte patterns (what an analyst would flag)

Machine-code idioms spotted in the bytes: shellcode stubs, PEB walks, anti-debug/anti-VM instructions, packer stubs, indirect control flow, embedded PEs and W+X sections — with file offsets.

Packed — 2 high / 4 medium / 2 info

Severity	Indicator	Count	Offsets / names	Why it matters
HIGH	int 2Dh anti-debug	1	`0x121a0`	int 2Dh — a well-known anti-debugging trap (behaves differently under a debugger)
HIGH	Known packer section name(s)	1	`.text1`	section names match a known packer/protector
MEDIUM	rdtsc timing check	2	`0x1b2a`, `0x1b2e`	reads the CPU timestamp counter — common timing-based anti-debug/anti-sandbox
MEDIUM	cpuid instruction	5	`0xbcdc`, `0xe8ac`, `0x1b226`, `0x263ad`, `0x2645a`	queries CPU/hypervisor features — common anti-VM (hypervisor-bit / vendor string) check
MEDIUM	pushad/popad packer stub	374	`0x30b`, `0x2445`, `0x3348`, `0x42b6`, `0x443e`, `0x6344`	save-all/restore-all register bracket — hallmark of a packer unpacking stub
MEDIUM	High-entropy (likely compressed/encrypted) section(s)	1	`.text1 (H=7.958)`	entropy >= 7.0 indicates packed/encrypted content (hidden payload)
INFO	Indirect call through register (call eax..edi)	53	`0x1a26`, `0x1ad9`, `0x1df1`, `0x1e52`, `0x1ea6`, `0x1f01`	dynamically-resolved call target — obfuscated control flow / runtime API resolution
INFO	Indirect jump through register (jmp eax..edi)	17	`0x9d40`, `0xcdae`, `0xd77c`, `0xe7a8`, `0x13b8a`, `0x13f7a`	computed jump — tail-call into unpacked code / obfuscated control flow

Flagged instructions at/near entry (0x100033f3, x86):

```
0x10003476: call esi   ; indirect call (computed/obfuscated target)
0x10003497: call ecx   ; indirect call (computed/obfuscated target)
0x100034bb: int3   ; breakpoint/anti-disasm
```

```
0x100033f3: lea esi, [0x10003414]
0x100033f9: mov dword ptr [0x10005010], esp
0x100033ff: mov dword ptr [0x10005008], ebx
0x10003405: push esi
0x10003406: jmp 0x10003409
0x10003408: ret
0x10003409: mov dword ptr [0x1000500c], ebp
0x1000340f: jmp 0x10003408
0x10003411: mov dword ptr [ebp - 4], eax
0x10003414: push ebp
0x10003415: mov ebp, esp
0x10003417: push esi
0x10003418: sub esp, 0x28
0x1000341b: mov dword ptr [ebp - 8], 0x47e5a87e
0x10003422: mov dword ptr [ebp - 0xc], 0x114b2e7f
0x10003429: call 0x10002879
0x1000342e: mov ecx, 0x47e5a87e
0x10003433: sub ecx, dword ptr [ebp - 8]
0x10003436: cmp eax, ecx
0x10003438: jne 0x100034a1
0x1000343a: mov eax, 0x550a76b
0x1000343f: cmp eax, dword ptr [ebp - 0xc]
0x10003442: jae 0x10003480
0x10003444: lea eax, [0x10004442]
0x1000344a: xor ecx, ecx
0x1000344c: mov edx, 1
0x10003451: mov esi, esp
0x10003453: mov dword ptr [esi + 8], 1
0x1000345a: mov dword ptr [esi + 4], 0
0x10003461: mov dword ptr [esi], 0x10004442
0x10003467: mov esi, dword ptr [0x1000400c]
0x1000346d: mov dword ptr [ebp - 0x10], eax
0x10003470: mov dword ptr [ebp - 0x14], ecx
0x10003473: mov dword ptr [ebp - 0x18], edx
0x10003476: call esi
0x10003478: sub esp, 0xc
0x1000347b: cmp eax, 0
0x1000347e: jne 0x100034a1
0x10003480: lea eax, [0x10004456]
0x10003486: mov ecx, esp
```

PE sections (anatomy):

Name	VSize	RawSize	Entropy	Flags
`.text`	9,506	12,288	5.548	X
`.rdata`	3,204	4,096	3.211	-
`.data`	4,292	4,096	0.08	W
`.text1`	179,563	180,224	7.958	W
`.rsrc`	1,816	4,096	1.322	-
`.reloc`	276	4,096	0.665	-

Unpacked — 0 high / 4 medium / 3 info

Severity	Indicator	Count	Offsets / names	Why it matters
MEDIUM	rdtsc timing check	1	`0x25afe`	reads the CPU timestamp counter — common timing-based anti-debug/anti-sandbox
MEDIUM	int3 (0xCC) sled	280	`0x1012`, `0x1042`, `0x1069`, `0x1082`, `0x10c1`, `0x13ca`	run of int3 bytes — padding/anti-disassembly or a breakpoint trap
MEDIUM	pushad/popad packer stub	92	`0x1ff`, `0x102c`, `0x1351`, `0x16cf`, `0x1a43`, `0x1f4b`	save-all/restore-all register bracket — hallmark of a packer unpacking stub

Severity	Indicator	Count	Offsets / names	Why it matters
MEDIUM	High-entropy (likely compressed/encrypted) section(s)	2	`.rdata (H=7.982)`, `.data (H=7.631)`	entropy >= 7.0 indicates packed/encrypted content (hidden payload)
INFO	Indirect call through register (call eax..edi)	50	`0x1767`, `0x5bdd`, `0x5bf2`, `0x5c07`, `0x5c1c`, `0x5e45`	dynamically-resolved call target — obfuscated control flow / runtime API resolution
INFO	Indirect jump through register (jmp eax..edi)	6	`0x62e2`, `0x64ee`, `0xb673`, `0x1b977`, `0x1c867`, `0x229c3`	computed jump — tail-call into unpacked code / obfuscated control flow
INFO	Writable self-reference (GetModuleHandle(NULL) pattern)	2	`0x629e`, `0x6383`	push 0 ; call [import] — often GetModuleHandleA(NULL) before manual mapping

Flagged instructions at/near entry (0x100033f3, x86):

```
0x10003463: int3   ; breakpoint/anti-disasm
0x100034a3: int3   ; breakpoint/anti-disasm

0x100033f3: add byte ptr [edx], ch
0x100033f6: lea ecx, [ebp - 8]
0x100033f9: call 0x10017380
0x100033fe: xor edx, edx
0x10003400: lea ecx, [ebp - 0x1c]
0x10003403: call 0x100073a0
0x10003408: mov eax, dword ptr [eax]
0x1000340a: push eax
0x1000340b: lea ecx, [ebp - 0x10]
0x1000340e: call 0x10017ec0
0x10003413: lea ecx, [ebp - 0x1c]
0x10003416: call 0x10016d40
0x1000341b: mov edx, 1
0x10003420: lea ecx, [ebp - 0x1c]
0x10003423: call 0x100073a0
0x10003428: mov ecx, dword ptr [eax]
0x1000342a: push ecx
0x1000342b: lea ecx, [ebp - 8]
0x1000342e: call 0x10017ec0
0x10003433: lea ecx, [ebp - 0x1c]
0x10003436: call 0x10016d40
0x1000343b: xor esi, esi
0x1000343d: lea ecx, [ecx]
0x10003440: push 0xd79959
0x10003445: push 0x2c05fbca
0x1000344a: mov dword ptr [ebp - 0x14], 0
0x10003451: call 0x1000d0d0
0x10003456: test eax, eax
0x10003458: je 0x10003465
0x1000345a: lea edx, [ebp - 0x14]
0x1000345d: push edx
0x1000345e: push 0x10002de0
0x10003463: int3
0x10003464: ret
0x10003465: cmp dword ptr [ebp - 0x14], 0
0x10003469: jne 0x10003488
0x1000346b: mov ecx, 0x3e8
0x10003470: call 0x1000ee30
0x10003475: inc esi
0x10003476: cmp esi, 0x3e8
```

PE sections (anatomy):

Name	VSize	RawSize	Entropy	Flags
`.text`	134,417	134,417	6.552	X
`.rdata`	24,318	24,318	7.982	-
`.data`	1,000	1,000	7.631	W
`.reloc`	3,784	3,784	3.88	-

7 · LOAD — How the sample was started under the debugger

- Module `178ba564b39bd07577e974a9b677dfd86ffa1f1d0299dfd958eb883c5ef6c3e1_d2df96.exe`, entry `0x100033F3`, first stop at `0x771D4DF1`.
- **paused early in the loader, before any of the sample's code ran**

  The debugger did not stop at the entry point (it can stop first inside the runtime or the loader). Code that ran before that first stop was not observed; the API arsenal was armed from the first stop onward.

8 · INTEL — Detection package (for AV, firewall, EDR and threat-intel vendors)

Files are in `intel/`: **35** indicators (**30** high-confidence, tier >= 70), 0 filtered out as known-good/noise (listed with reasons in `iocs.json`), 0 Suricata rule(s), 0 Sigma rule(s), STIX 2.1 bundle: yes.

Indicator class	Found	Not available because
file hashes	9	sample + unpacked payloads
network (C2 / download)	0 (0 high-confidence)	none recovered with confidence: the C2 configuration is probably encrypted at rest and decrypted only when it beacons — run the detonation phase (FakeNet capture) or recover the config key
host artefacts	0	none observed in the code recovered so far
build / attribution	5	

YARA rules (one per stage):

Stage	File	Result
packed	`178ba564b39bd07577e974a9b677dfd86ffa1f1d0299dfd958eb883c5ef6c3e1.exe`	written — 12 strings, needs 6; goodware false positives 0/28
unpacked	`api_fullmodule_0x10000000_rebuilt.exe`	written — 12 strings, needs 6; goodware false positives 0/28
unpacked	`api_ntwx_0x10001000.bin`	withheld: only 1 distinctive strings in this stage (need >= 4) — a rule would be noisy

Tiers: observed 90 · decrypted 80 · unpacked 70 · static 40 · AI-suggested 20 (never emitted as a rule). Auto-generated YARA must be validated against your goodware set before deployment.

2. Static analysis — what each tool discovered

2.1 Detect It Easy (identification)

- File type: **PE32**
- Compiler/runtime: **Microsoft Visual C/C++ 19.00.24215**
- Linker: Microsoft Linker 9.00.21022
- Packer (signature): none matched (custom/unknown)

2.2 PE profile (pefile) — the PE Studio data

- Architecture: x86 | kind: exe | entry point: 0x33f3 in `.text`
- Imports: 17 functions / 10 DLLs | TLS callbacks: 0 | signed: False | overlay: 0 bytes
- Sections (name / flags / entropy):
  - `.text` X- entropy 5.55 <entry
  - `.rdata` -- entropy 3.21
  - `.data` -W entropy 0.08
  - `.text1` -W entropy 7.96
  - `.rsrc` -- entropy 1.32
  - `.reloc` -- entropy 0.66

2.3 Ghidra (disassembly / code review)

- unavailable: None

2.4 de4dot (.NET deobfuscation)

- not a .NET assembly (native path)

3. Memory dumps — hashes, comparison & re-static

Packed input: MD5 `6a8401448a5bd2b540850f811b20a66d` · SHA-256 `178ba564b39bd07577e974a9b677dfd86ffa1f1d0299dfd958eb883c5ef6c3e1` · 17 imports.

Every dumped region, with its own hashes (copy to compare / VirusTotal) and what re-static found in it:

Dump (link)	Size	PE	Imports	re-static URLs/IPs	Same as packed?	MD5	SHA-256
api_fullmodule_0x10000000_PE.bin	217,088	yes	7	0/0	no	8e83c0e987ab924696ecbf13b7d380fa	73e2d956e6e712eee57d85215ca81ae5fd4db2cc0802038672c08790c3491029
api_fullmodule_0x10000000_rebuilt.exe	217,088	yes	7	0/0	no	9fc8d8fc35a3e4eb385f169eb16d6d16	bc0886d626ba6ca8c60ab997b578c66a92abae0a72fa1a17fbe4cd0570892289
api_ntwx_0x10001000.bin	134,417	-	-	0/0	no	691ceb869f2c5120a139d766242ec0f4	e7414cb2abfd3684187e96c4fb7b6426689e48fe96c5a520e474fe440d3491de
api_plan_7000.bin	179,563	-	-	0/0	no	0a2fdfae075f7b1f75c66c361b7b3a75	4a70fa518fa190dd0e222e8248527d4eed1d326de7fdf979abeca1ed90a0d678
api_region0_0x670000.bin	176,128	-	-	0/0	no	2740f5927c507bc0df49f3a70d387699	1049554dd71543ea961666920a5d58bc10bc3545480442 17c5f1556385edbebe
api_region1_0x1e0000_PE.bin	24,576	yes	2	0/0	no	b8dd20ffc4818f4d84926096b7fb5a0c	21d45d1d7805dc08a41fc906b59c9bc1282dc71c55c272d512bd600374605b6c

Dump (link)	Size	PE	Imports	re-static URLs/IPs	Same as packed?	MD5	SHA-256
api_region1_0x1e0000_rebuilt.exe	24,576	yes	2	0/0	no	a82f9ad175ac3348c41732ab35a8ae37	21371fb6006741a1e36c5b09a8fcdc6006f71f0dbb7c2e79e29b2db5f357293c
api_region2_0x6f0000.bin	172,032	-	-	0/0	no	72389e9e2971cd7227dd5aa2543d6c73	2cbbeef1249170a43854962fa5b19fba628470c70beb9ce23e15a0f05cb891f2
api_wx_0x10001000.bin	134,417	-	-	0/0	no	691ceb869f2c5120a139d766242ec0f4	e7414cb2abfd3684187e96c4fb7b6426689e48fe96c5a520e474fe440d3491de

Packed vs reconstructed — did unpacking change the file?

	Packed input	Reconstructed payload
MD5	6a8401448a5bd2b540850f811b20a66d	9fc8d8fc35a3e4eb385f169eb16d6d16
Imports	17	7
Sections	['.text', '.rdata', '.data', '.text1', '.rsrc', '.reloc']	['.text', '.rdata', '.data', '.reloc']
Different file?	—	YES (distinct content)

3c. Dump-by-dump analysis (what each dump actually is, from its bytes)

9 dump(s), 8 distinct. Classified by CONTENT, not by file name.

#	Dump	Size	What it is	Entropy	Why
1	api_wx_0x10001000.bin	134,417	**executable code**	6.552	100% of the first bytes decode as valid x86 instructions: looks like shellcode or a code region
2	api_fullmodule_0x10000000_PE.bin	217,088	**PE image**	5.744	valid PE header; 7 imports across 2 DLL(s); still packed-looking (high-entropy section or almost no imports)
3	api_ntwx_0x10001000.bin *(duplicate of api_wx_0x10001000.bin)*	134,417	**executable code**	6.552	100% of the first bytes decode as valid x86 instructions: looks like shellcode or a code region
4	api_region0_0x670000.bin	176,128	**zero-filled**	0.455	96% zero bytes: memory the program allocated but had not (yet) filled
5	api_region1_0x1e0000_PE.bin	24,576	**PE image**	2.503	valid PE header; 2 imports across 1 DLL(s); still packed-looking (high-entropy section or almost no imports)
6	api_region2_0x6f0000.bin	172,032	**zero-filled**	-0.0	100% zero bytes: memory the program allocated but had not (yet) filled
7	api_plan_7000.bin	179,563	**executable code**	5.893	100% of the first bytes decode as valid x86 instructions: looks like shellcode or a code region
8	api_fullmodule_0x10000000_rebuilt.exe	217,088	**PE image**	5.744	valid PE header; 7 imports across 2 DLL(s); still packed-looking (high-entropy section or almost no imports)
9	api_region1_0x1e0000_rebuilt.exe	24,576	**PE image**	2.503	valid PE header; 2 imports across 1 DLL(s); still packed-looking (high-entropy section or almost no imports)

PE images:

- `api_fullmodule_0x10000000_PE.bin`: 7 imports / 2 DLLs (KERNEL32.dll, ADVAPI32.dll), entry `0x33f3`, imphash `eb76140cc31cd615e218e51dbcad0a7b`, sections ['.text', '.rdata', '.data', '.reloc'], high-entropy sections ['.rdata', '.data']
- `api_region1_0x1e0000_PE.bin`: 2 imports / 1 DLLs (KERNEL32.dll), entry `0x1e97`, imphash `d073d1854680ff2ddd038760f5090798`, sections []
- `api_fullmodule_0x10000000_rebuilt.exe`: 7 imports / 2 DLLs (KERNEL32.dll, ADVAPI32.dll), entry `0x33f3`, imphash `eb76140cc31cd615e218e51dbcad0a7b`, sections ['.text', '.rdata', '.data', '.reloc'], high-entropy sections ['.rdata', '.data']
- `api_region1_0x1e0000_rebuilt.exe`: 2 imports / 1 DLLs (KERNEL32.dll), entry `0x1e97`, imphash `d073d1854680ff2ddd038760f5090798`, sections []

How consecutive stages relate:

From	To	Same bytes at same offsets	Shared blocks	Entropy change	New strings	Reading
`api_wx_0x10001000.bin`	`api_fullmodule_0x10000000_PE.bin`	3%	32	-0.81	93	api_fullmodule_0x10000000_PE.bin contains all of api_wx_0x10001000.bin (a larger capture of the same memory)
`api_fullmodule_0x10000000_PE.bin`	`api_region0_0x670000.bin`	15%	0	-5.29	8	15% of bytes match at the same offsets; partly related
`api_region0_0x670000.bin`	`api_region1_0x1e0000_PE.bin`	63%	0	+2.05	0	the region was empty in api_region0_0x670000.bin and filled with PE image by api_region1_0x1e0000_PE.bin
`api_region1_0x1e0000_PE.bin`	`api_region2_0x6f0000.bin`	74%	0	-2.5	0	74% of bytes match at the same offsets; partly related
`api_region2_0x6f0000.bin`	`api_plan_7000.bin`	27%	0	+5.89	105	the region was empty in api_region2_0x6f0000.bin and filled with executable code by api_plan_7000.bin
`api_plan_7000.bin`	`api_fullmodule_0x10000000_rebuilt.exe`	10%	35	-0.15	7	api_fullmodule_0x10000000_rebuilt.exe contains all of api_plan_7000.bin (a larger capture of the same memory)
`api_fullmodule_0x10000000_rebuilt.exe`	`api_region1_0x1e0000_rebuilt.exe`	22%	0	-3.24	8	22% of bytes match at the same offsets; partly related imports 7 -> 2

3d. Malware category

No category is supported by the evidence from this run. If the sample is a packer or loader, the category belongs to the unpacked payload.

3e. File-encryption analysis

No file-encryption call was observed in this run.

3b. Detonation memory dump + re-static

- No full-process dump captured (process exited quickly or injected elsewhere).
- **PE-sieve carved 2 PE module(s) from process memory:**
  - `10000000.malware.exe` (161 KB)
  - `510000.dll` (9 KB)

4. Dynamic analysis (B) — live detonation (observed behavior)

- Ran 90s, isolated (FakeNet contained all network). Completed: True

- **Network (FakeNet):** 0
- **Registry changes (Regshot diff):** 0
- **Files dropped/created:** 0

4a. What the sample did under Procmon (438,198 events from `malware.exe`)

Operation	Count
RegQueryValue	172,357
RegQueryKey	119,333
RegOpenKey	66,419
RegCloseKey	39,802
RegSetInfoKey	39,796
Process Profiling	116
CreateFile	73
QuerySecurityFile	57
IRP_MJ_CLOSE	52
CloseFile	50
Load Image	32
QueryOpen	28

Busiest registry locations (reads):

- `HKLM` (119,314x)
- `HKLM\SOFTWARE\WOW6432Node\Microsoft\Cryptography\Defaults\Provider\Microsoft Enhanced RSA and AES Cryptographic Provider` (79,530x)
- `HKLM\SOFTWARE\WOW6432Node\Microsoft\Cryptography\Defaults\Provider Types\Type 024` (66,276x)
- `HKLM\SOFTWARE\Microsoft\Cryptography` (53,020x)
- `HKLM\Software\Microsoft\Cryptography` (13,255x)
- `HKLM\Software\WOW6432Node\Microsoft\Cryptography\Offload` (13,255x)
- `HKLM\Software\WOW6432Node\Microsoft\Cryptography\DESHashSessionKeyBackward` (13,255x)
- `HKLM\System\CurrentControlSet\Control\Session Manager` (19x)

Writes / creates: none observed
Processes created: none observed
Network operations: none observed

5. Comparison — static vs memory vs live detonation

Indicator source	URLs	IPs	Registry	Notes
Static (on-disk)	0	0	-	limited (packed/obfuscated)
Memory dump (re-static)	0	0	0	decrypted at runtime
Live detonation	0	-	0	observed behavior

Interpretation: indicators the on-disk file hides (because it is packed/obfuscated) surface once the sample decrypts itself in memory and acts on the network — which is why the memory dump and live detonation are decisive.

6. MITRE ATT&CK

- T1027.002 Software Packing
- T1140 Deobfuscate/Decode Files or Information
- T1620 Reflective Code Loading

7. Artifacts

- Analysis folder: `ce45ac67-2b3c-455d-9c12-e11b0a0c5059_dridex/` (subfolders: dumps/, source/, decryptor/, logs/, artifacts/)
- Detonation artifacts: `cases/ce45ac67-2b3c-455d-9c12-e11b0a0c5059_dridex/artifacts/run` (memory .dmp, FakeNet pcap/console, Procmon .pml/.csv, reg snapshots, file lists)

7b. Tool audit (per-subtool reliability)

- Total subtool calls: **405** (386 ok, 19 failed) in 218420 ms.

Tool	Calls	Errors	Time (ms)
ssh.run	39	2	119185
scp.pull	8	0	73230
de4dot	1	0	8293
diec	6	0	5779
scp.push	3	0	5289
x64dbg.eval_expression	113	0	1500
x64dbg.debug_get_state	99	0	1394
x64dbg.script_execute	87	0	1383
x64dbg.module_get_exports	18	15	1185
x64dbg.debug_pause	1	0	571
pefile	6	0	173
x64dbg.dump_memory_region	6	0	162
x64dbg.module_get_main	5	1	63
x64dbg.register_get_batch	3	0	45
x64dbg.breakpoint_set	3	0	44
x64dbg.memory_read	3	1	42
x64dbg.dump_module	1	0	39
x64dbg.debug_stop	2	0	26
x64dbg.debug_init	1	0	17

Failed calls (trace these first):

- #3 x64dbg.module_get_main — {'code': -32000, 'message': 'Failed to get main module info'}
- #6 ssh.run — ssh returncode=128
- #131 x64dbg.module_get_exports — {'code': -32603, 'message': 'Module not found: mpr.dll'}
- #132 x64dbg.module_get_exports — {'code': -32603, 'message': 'Module not found: netapi32.dll'}
- #133 x64dbg.module_get_exports — {'code': -32603, 'message': 'Module not found: urlmon.dll'}
- #134 x64dbg.module_get_exports — {'code': -32603, 'message': 'Module not found: winhttp.dll'}
- #135 x64dbg.module_get_exports — {'code': -32603, 'message': 'Module not found: ws2_32.dll'}
- #192 x64dbg.memory_read — {'code': -32010, 'message': 'Memory not readable at address: 0x001E30F2'}
- #213 x64dbg.module_get_exports — {'code': -32603, 'message': 'Module not found: mpr.dll'}
- #214 x64dbg.module_get_exports — {'code': -32603, 'message': 'Module not found: netapi32.dll'}
- #215 x64dbg.module_get_exports — {'code': -32603, 'message': 'Module not found: urlmon.dll'}
- #216 x64dbg.module_get_exports — {'code': -32603, 'message': 'Module not found: winhttp.dll'}
- #217 x64dbg.module_get_exports — {'code': -32603, 'message': 'Module not found: ws2_32.dll'}
- #302 x64dbg.module_get_exports — {'code': -32603, 'message': 'Module not found: mpr.dll'}
- #303 x64dbg.module_get_exports — {'code': -32603, 'message': 'Module not found: netapi32.dll'}
- #304 x64dbg.module_get_exports — {'code': -32603, 'message': 'Module not found: urlmon.dll'}
- #305 x64dbg.module_get_exports — {'code': -32603, 'message': 'Module not found: winhttp.dll'}
- #306 x64dbg.module_get_exports — {'code': -32603, 'message': 'Module not found: ws2_32.dll'}
- #372 ssh.run — ssh returncode=1
- Full per-call ledger (inputs, output, timing): logs/tool_audit.jsonl

8. Notes & limitations

- Detonation ran in an isolated VM with no real internet (FakeNet). Revert the snapshot after.
- Network/OS/tooling noise is filtered from the indicators above.

Appendix A — Methodology & reproducibility

Reproducibility & Methodology — 178ba564b39bd07577e974a9b677dfd86ffa1f1d0299dfd958eb883c5ef6c3e1.exe

This report is deterministic: the same sample + the same tool versions + the clean VM baseline reproduce these findings. Below is the exact flow, the command/step counts, and a per-finding replay recipe.

1. Environment fingerprint (pin these to reproduce)

- Sample SHA-256: `178ba564b39bd07577e974a9b677dfd86ffa1f1d0299dfd958eb883c5ef6c3e1`
- Size: 212,992 bytes | type: PE32 (x86)
- Detect It Easy: PE32 / compiler Microsoft Visual C/C++ 19.00.24215 (diec 3.21)
- Ghidra: analyzeHeadless (NemesisStatic.java) | pe-sieve 0.4.1.1 | upx (brew)
- Debugger: x64dbg 2026 build (C:\x64dbg_new), x64dbg-mcp plugin v1.0.11, MCP 127.0.0.1:3000
- VM: revert to `clean-baseline` snapshot before each run (identical start state)

2. Analysis flow (ordered — each stage is recorded)

1. **Detect It Easy + pefile** (`triage`) — diec -j ; pefile parse
2. **author/build recon** (`attribution`) — pefile: TimeDateStamp, debug PDB, Rich header, version info
3. **unpack verification** (`verify_unpack`) — diff packed vs unpacked (imports/sections/entropy/IOCs)
4. **.NET deobfuscation** (`guest_deobfuscation`) — DIE + de4dot (guest)
5. **x64dbg OEP unpack (MCP)** (`dynamic_unpack`) — restart -> bph OEP -> run -> dump_module
6. **behavioral detonation** (`detonation`) — FakeNet + Procmon + Regshot + pe-sieve
7. **behavioral classification** (`classification`) — evidence-weighted family rules
8. **crypto detect + decryptor** (`crypto_recovery`) — scheme + key detection -> decrypt_.py

3. Tool-call statistics (from the audit ledger — exact counts)

Every subtool call is recorded in `logs/tool_audit.jsonl` with inputs, output, duration and status. Totals:

- **405 calls**, 386 ok, 19 failed, 218420 ms

Tool / command	Calls	Errors	Time (ms)
x64dbg.eval_expression	113	0	1500
x64dbg.debug_get_state	99	0	1394
x64dbg.script_execute	87	0	1383
ssh.run	39	2	119185
x64dbg.module_get_exports	18	15	1185
scp.pull	8	0	73230
pefile	6	0	173
diec	6	0	5779
x64dbg.dump_memory_region	6	0	162
x64dbg.module_get_main	5	1	63
scp.push	3	0	5289
x64dbg.breakpoint_set	3	0	44
x64dbg.register_get_batch	3	0	45
x64dbg.memory_read	3	1	42
x64dbg.debug_stop	2	0	26
x64dbg.debug_init	1	0	17
x64dbg.debug_pause	1	0	571
x64dbg.dump_module	1	0	39
de4dot	1	0	8293

4. Reproduce the STATIC findings

```
diec -j <sample>                # identification: PE32
python3 -c 'import pefile; ...'  # sections, imports, entropy, TLS, signature
```

4b. Step-by-step execution (what the sample did, in order)

Addresses differ every run (ASLR) and across machines, so each step is given **relative** to the module base (RVA = address − base) and by **call order / arguments**, which are invariant. Follow these in any debugger regardless of where the module loads.

1. **1× VirtualAlloc** — lpAddress=0x0, dwSize=0x2b000, flType=0x1000 → returned base 0x670000
2. **2× VirtualAlloc** — lpAddress=0x0, dwSize=0x6000, flType=0x1000 → returned base 0x1e0000
3. **1× LoadLibraryA** — lpName=0x1e30f2 → ""
4. **3× VirtualAlloc** — lpAddress=0x0, dwSize=0x2a000, flType=0x1000 → returned base 0x6f0000
5. **2× LoadLibraryA** — lpName=0x717e80 → "KERNEL32.dll"
6. **3× LoadLibraryA** — lpName=0x717e9e → "ADVAPI32.dll"
7. **1× VirtualProtect** — lpAddress=0x10000000, dwSize=0x35000, flNewProtect=0x4 → protect 0x4 (not exec)
8. **1× NtProtectVirtualMemory** — hProc=0xffffffff, pBase=0x19fcf8, pSize=0x19fcfc → NtProtect newProt=0x4
9. **2× VirtualProtect** — lpAddress=0x10000000, dwSize=0x400, flNewProtect=0x2 → protect 0x2 (not exec)
10. **2× NtProtectVirtualMemory** — hProc=0xffffffff, pBase=0x19fcf8, pSize=0x19fcfc → NtProtect newProt=0x2
11. **3× VirtualProtect** — lpAddress=0x10001000, dwSize=0x20d11, flNewProtect=0x20 → W->X region 0x10001000+0x20d11 [full-module dumped]
12. **3× NtProtectVirtualMemory** — hProc=0xffffffff, pBase=0x19fcf8, pSize=0x19fcfc → W->X (native) 0x10001000+0x20d11
13. **4× VirtualProtect** — lpAddress=0x10022000, dwSize=0x5efe, flNewProtect=0x2 → protect 0x2 (not exec)
14. **4× NtProtectVirtualMemory** — hProc=0xffffffff, pBase=0x19fcf8, pSize=0x19fcfc → NtProtect newProt=0x2
15. **5× VirtualProtect** — lpAddress=0x10028000, dwSize=0x3e8, flNewProtect=0x4 → protect 0x4 (not exec)
16. **5× NtProtectVirtualMemory** — hProc=0xffffffff, pBase=0x19fcf8, pSize=0x19fcfc → NtProtect newProt=0x4
17. **6× VirtualProtect** — lpAddress=0x10029000, dwSize=0xec8, flNewProtect=0x2 → protect 0x2 (not exec)
18. **6× NtProtectVirtualMemory** — hProc=0xffffffff, pBase=0x19fcf8, pSize=0x19fcfc → NtProtect newProt=0x2

(module preferred base `0x10000000`; at runtime read the real base from the memory map and add the RVA. The payload W→X flip + its size are the anchor.)

5. Reproduce the DYNAMIC unpack (x64dbg, by hand)

Load the sample in x32dbg (paused at entry), then:

```
bc ; bphwc                      # clear software + hardware breakpoints
run           # repeat; use 'erun' to pass first-chance exceptions
#   -> stops at OEP None
# dump: dump_module(auto_detect_oep=true, rebuild_pe=true)  [mode: None, new_ep: None]
```

7. Per-finding provenance (where each IOC came from)

Indicator	Source stage	How to re-derive
(no network IOCs recovered)	—	—

8. Determinism — what is and isn't fixed

- **Deterministic:** identification, PE structure, imports, Ghidra functions/anomalies, recovered source, detected OEP (RVA), API call counts, and the dumped payload's content — all reproduce exactly from the same sample.
- **Environment-fixed:** module base may differ with ASLR; we record the runtime base so addresses map (RVA is invariant). Revert to `clean-baseline` for an identical OS.

- **Non-deterministic (noted, not relied on):** wall-clock timings in the ledger, and any network the sample attempts (contained by the isolated VM / FakeNet).
- **Audit trail:** `logs/tool_audit.jsonl` is the full machine-readable record; every row has the tool, inputs, output summary, duration and status.

Appendix B — Proof of compromise

Proof of Compromise — 178ba564b39bd07577e974a9b677dfd86ffa1f1d0299dfd958eb883c5ef6c3e1.exe

Verdict: MALICIOUS (confidence 65/100, static + dynamic + detonation)

Family: Behaves as a **loader/packer**.

Identity

- SHA-256: `178ba564b39bd07577e974a9b677dfd86ffa1f1d0299dfd958eb883c5ef6c3e1`
- Size: 212,992 bytes
- Type: PE32 (x86)
- Compiler: Microsoft Visual C/C++ 19.00.24215

Author / build reconnaissance

- Compiled 2020-04-24 02:04:11 UTC
- Version info: CompanyName=INCA Internet Co., Ltd., LegalCopyright=Copyright (C) INCA Internet. 2000-2003, Comments=nProtect KeyCrypt Program Database DLL, OriginalFilename=npkpdb.dll, ProductName=nProtect KeyCrypt Program Database DLL, InternalName=npkpdb.dll
- MSVC toolchain build IDs [21005, 21022, 23026, 23918, 24210, 24213, 24215, 25834, 30319, 30501, 30729, 31101, 40219, 40629, 50727, 60610, 61030] (Rich-header fingerprint for same-author clustering)

Why (evidence)

- CONFIRMED self-unpacking at runtime (payload captured + verified) (+30)
- runtime: memory made executable after being written (2 W->X event(s)) — the unpacking moment (+10)
- behavioral classification: loader/packer (loader/packer) (+25)

Unpack verification

- **CONFIRMED_UNPACKED** (confidence 60/100) — the dumped/unpacked payload was compared against the packed file.
  - debugger captured the payload at 2 write->execute flip(s) (direct unpack evidence)
  - dump parses as a valid PE image

Packing

- Packed: not conclusively (confidence 10/100)
  - uncommon high-entropy section '.text1' (7.96)

MITRE ATT&CK

- T1027.002 Software Packing
- T1140 Deobfuscate/Decode Files or Information
- T1620 Reflective Code Loading

Deobfuscation (.NET)

- .NET: False

Detonation (behavioral)

- started: 2026-10-21T16:22:18
- duration_s: 90
- artifacts_dir: cases/ce45ac67-2b3c-455d-9c12-e11b0a0c5059_dridex/artifacts/run